

COMPENG 2DX3

Final Report

Cameron Lin, 400535393

Date: April 8th, 2025

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

Contents

Device Overview	3
Features	3
TI MSP-EXP432E401Y.....	3
Onboard Peripherals.....	3
VL53L1X Time of Flight Sensor.....	3
28BYJ-48 Stepper Motor.....	4
Python 3D Visualization	4
General Description	5
Block Diagram	6
Device Characteristics	6
Detailed Description	7
Distance Measurement	7
Visualization	8
Limitations	14
Floating Point Capability	14
Quantization Error	14
Speed Limitations	14
Required Bus Speed.....	14
Circuit Schematic	15
Programming Logic Flow	16

Device Overview

Features

TI MSP-EXP432E401Y

- Core Microcontroller, controlling sensors and acting as the main connection between sensors and the computer for data visualization
- Uses a 32-bit Cortex M4F CPU at bus speed of 22 MHz but can go up to 120 MHz
- 1 MB of flash memory, 256 kB of SRAM
- Onboard buttons and LEDs used for control and status

Onboard Peripherals

- PJ0 activates/deactivates the time-of-flight sensor
- PJ1 enables the motor, and returns the motor to home if pressed while rotating
- LED D3 (PF4) signifies whether time-of-flight sensor is on
- LED D2 (PN0) signifies if time-of-flight sensor is gathering distance data
- LED D1 (PN1) signifies successful serial communication transfer

VL53L1X Time of Flight Sensor

- Accurate up to 4 m
- Up to 50 Hz measuring frequency
- Operating voltage between 2.6 – 3.5V
- I2C communication up to 400 kHz (Fast mode) with programmable address

28BYJ-48 Stepper Motor

- 4 Phased controls
- Operating voltage between 5 – 12V
- 2048 steps for 360 degrees of rotation
- Powered and controlled by additional motor driver

Python 3D Visualization

- PySerial used to gather serial data from USB port using UART
- Open3D used to visualize points and mesh

General Description

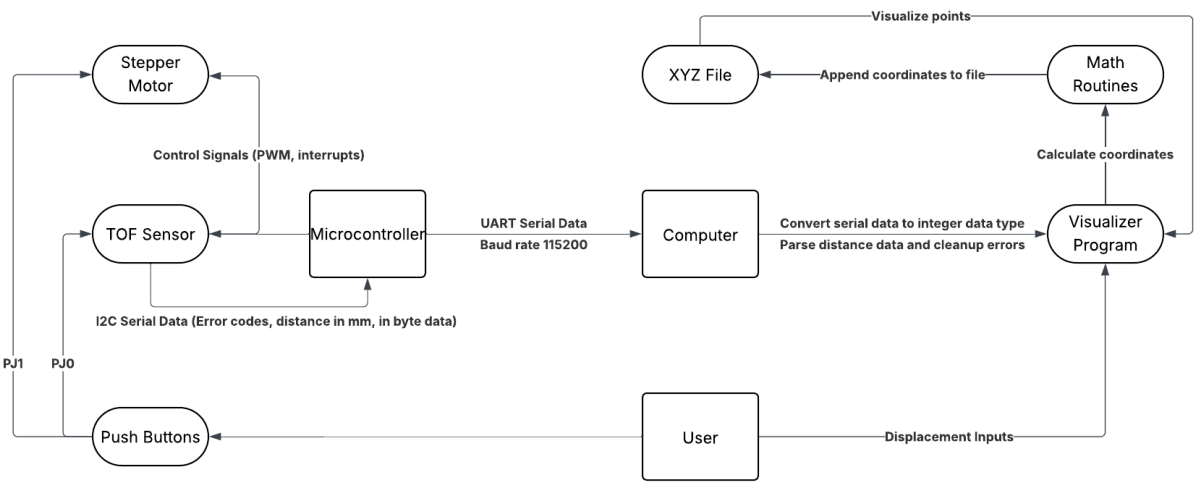
This device is an embedded system that provides a simple and low-cost solution to scan and visualize interiors in 3D. The included time-of-flight sensor can capture distances up to 4m every 5.625 degrees, leading to a highly detailed 64 points of accuracy for every slice captured (in the x-y plane). There are 2 onboard buttons for control, PJ0 enabling or disabling the time-of-flight sensor and PJ1 enabling the motor to complete one rotation.

The software component of this device lies in configuration of the code to meet the demands of the interior. In the python visualization code, you can configure the number of slices by adjusting the “depth” variable. You are also prompted to enter the amount of displacement (in mm) in the z-axis for further customization to allow for more detailed capturing of the interior.

A brief explanation of how the device works is as follows:

1. System waits for GPIO interrupts from onboard push buttons
2. Once time-of-flight and motor are switched on, motor starts rotating
3. Motor rotates 5.625 degrees
4. Wait for time-of-flight sensor to return distance data
5. Transmit data to microcontroller via I2C
6. Transmit data to computer via UART
7. Repeat step 3 until 64 steps are completed
8. Parse data in python file
9. Calculate x, y, z coordinates using trigonometry
10. Visualize points in Open3D

Block Diagram



Device Characteristics

Microcontroller	
Clock Speed	22 MHz
Push Buttons	PJ0, PJ1
Serial Port	Depends on device
UART	Pins configured for UART0 (vertical jumpers)
Baud Rate	115200 bps

Stepper Motor	
Vin	5 V
IN1	PH0
IN2	PH1
IN3	PH2
IN4	PH3

TOF Sensor	
Vin	3.3 V
SDA	PB3
SCL	PB2

Detailed Description

Distance Measurement

The distance measurement of this system uses the VL53L1X time-of-flight sensor. This sensor works by emitting an infrared laser pulse which is detected by the sensor. The VL53L1X automatically measures the time delay between the emission and reception of the pulse and calculates the distance using the formula:

$$Distance = \frac{time}{2} * speed\ of\ light$$

The way the distance measurement is performed is by using the stepper motor to rotate the time-of-flight sensor to gather distance measurements in a 360-degree area. The time-of-flight sensor is initialized in long-distance mode meaning it can measure up to 4m, at the cost of taking longer time to make measurements. The way the algorithm works is that the system counts the number of steps it takes to make a 5.625-degree rotation in the clockwise direction, and at that point calls the function to collect data. The program then waits in a loop to check if the time-of-flight sensor has gathered the data correctly and then moves on to transmit the data through the I2C connection. After the full 360 degree rotation has been completed, the motor rotates counter clockwise back to its home position to ensure that the wires do not get tangled.

When distance is captured, the microcontroller reads the sensor data through the I2C communication protocol and prints the data in the form of “RangeStatus, Distance (mm), SignalRate, AmbientRate, SpadNum”. This data is useful for debugging and making sure the sensor is working properly, however the main things we are looking for in the transmission are the RangeStatus which we want to be 0 (meaning no errors) and the distance.

Visualization

Once the data has been successfully transmitted to the microcontroller, it is then relayed to the host computer through UART at the user-specific COM port (COM5 in my case) at a baud rate of 115200 bps. Using the pySerial library, the UART transmission is encoded and then decoded into a string data type. The UART transmission is now in the form of a string like “RangeStatus, Distance (mm), SignalRate, AmbientRate, SpadNum”, which is then split into an array and separated by the comma character. The program is also in a loop which makes sure that error or bad transmissions are not accidentally inserted into the distance array. This works by putting the system into a while loop, and a try except block which tries to parse the data, and if it returns an error, request for another transmission. This loop also can read the error statement provided by the RangeStatus, and is configured to set the point to 4m if it is not 0, because an error in RangeStatus is likely to occur due to measuring a room that is too large.

Once the data is successfully parsed, converted to an integer and appended to an array, the program starts to calculate the x and y coordinates of the points. The following equations are used to calculate the distances:

$$x = \sin(\text{degree of rotation}) * \text{distance}$$

$$y = \cos(\text{degree of rotation}) * \text{distance}$$

After the distances are calculated, the coordinates are then written to a file in the .xyz format. Finally, the program uses the Open3D library which first reads the file and returns a point cloud object to the user. After it is closed, a line set is created which connects the points consecutively to create a 3D mesh. The 3D mesh is then rendered in an interactive window allowing the user to analyze and look around the area they mapped.

Example Use Case

Application Note

When programming this device, the following specifications were used:

- Asus ZenBook 15 UP3404
- Windows 11
- Core i7-1360P processor, integrated graphics
- 16 GB RAM

However, this setup should work on most devices that can run a Windows 10 or 11 operating system, or even more generally any device that has been released in the last 10-15 years.

When operating this device, it is optimal to place it in the center of the room, equidistant from each side of the room to allow for the most accurate data. It is also necessary to allow the full rotation of the motor to allow the program to work properly, as everything is coded with the built-in number of points in mind.

Setup

- 1) Plug microcontroller to computer via USB port
- 2) To figure out what COM port is being used, go to device manager and find the COM port under XDS110 Class... port.
- 3) Go to the python script and modify line 10 with 'COM5' to the one in device manager
- 4) Modify line 7, and the variable depth to how many slices of the room you would like to take
- 5) Also in the python script, make sure there is a file called test.xyz in the same folder as the script
- 6) Open the Kiel project and select the 2dx_studio_8c file
- 7) Click 'Translate' -> 'Build' -> 'Load', and press the reset button on the microcontroller
- 8) Press PJ0 to enable the time-of-flight sensor
- 9) Once the microcontroller is in the correct position, press launch the python script and press the enter key to start communication
- 10) Press PJ1 to start the motor and data transmission
- 11) Once the first slice has been measured, type in the displace of the microcontroller in mm
- 12) Repeat step 11 until all slices have been measured
- 13) Once finished, you should first see a point cloud, and then a 3D mesh of the room you just scanned

Expected Output

My assigned location for scanning was location D. The main features of this location are two-bathroom doors on the left side which create a sizable indent, a small bench on the right and a large hallway turning left at the end creating a large empty space.

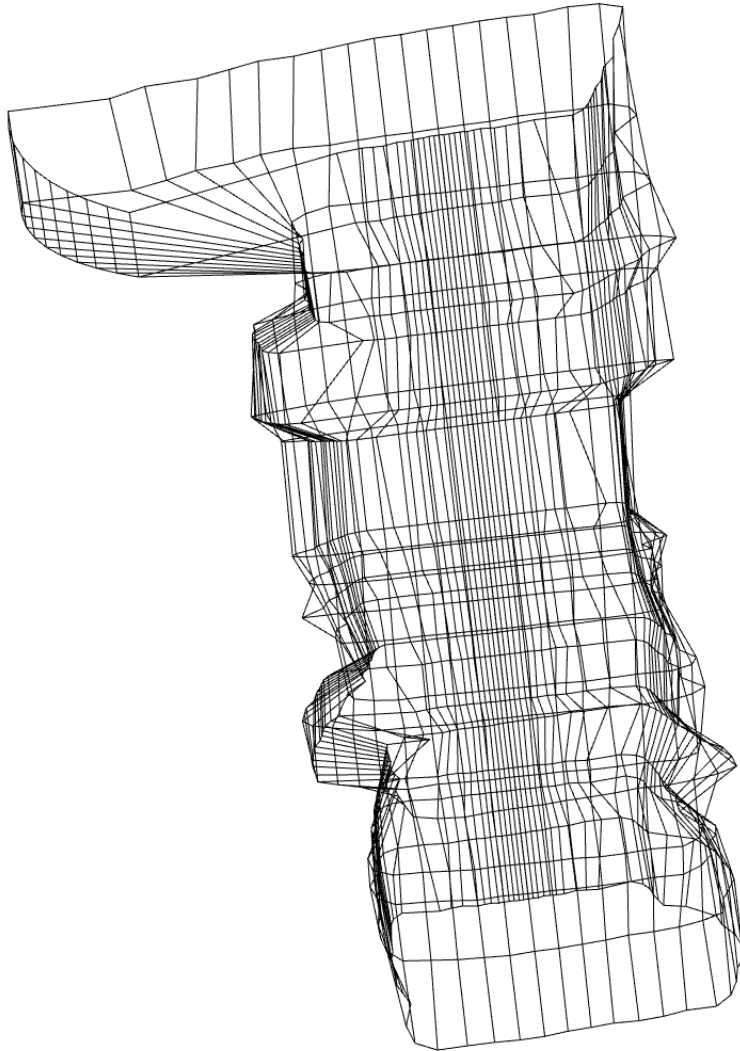


Figure 1: Birds Eye View of Location D

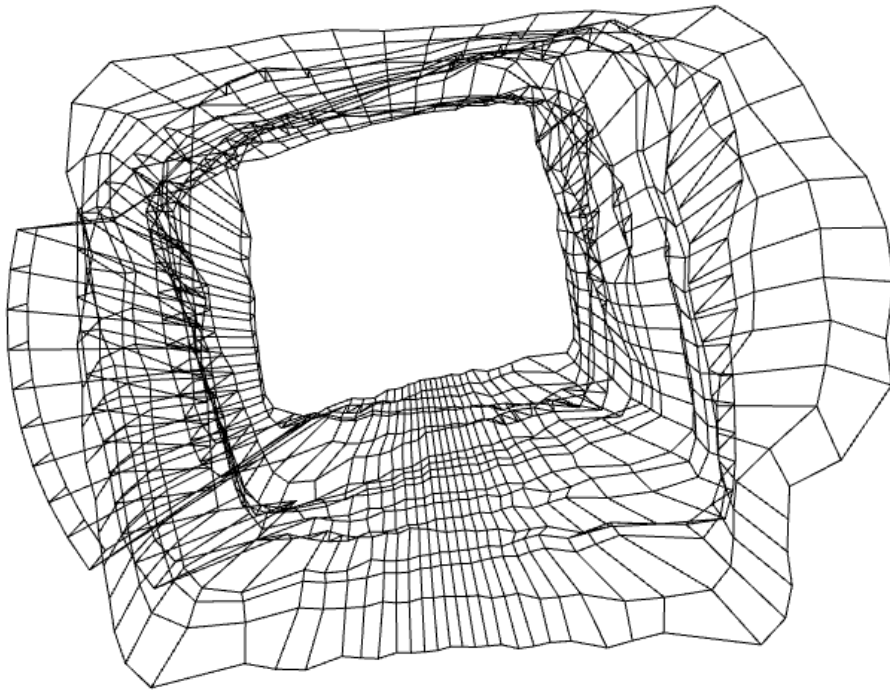


Figure 2: Frontal View of Location D



Figure 3: Real-Life photo of Location D

Limitations

Floating Point Capability

- The microcontroller used in this project has a 32-bit floating point unit that follows the IEEE754 format
- This means that it is possible to compute the points to a high degree of accuracy on the microcontroller itself
- The calculations were instead performed on a more powerful desktop instead due to simpler and more robust alternatives (python)

Quantization Error

- The microcontroller has a 16-bit ADC
- With distance being between 0-4000mm, the maximum quantization error for each measurement is:

$$Q = \frac{4000}{2^{16}} = 0.0610mm$$

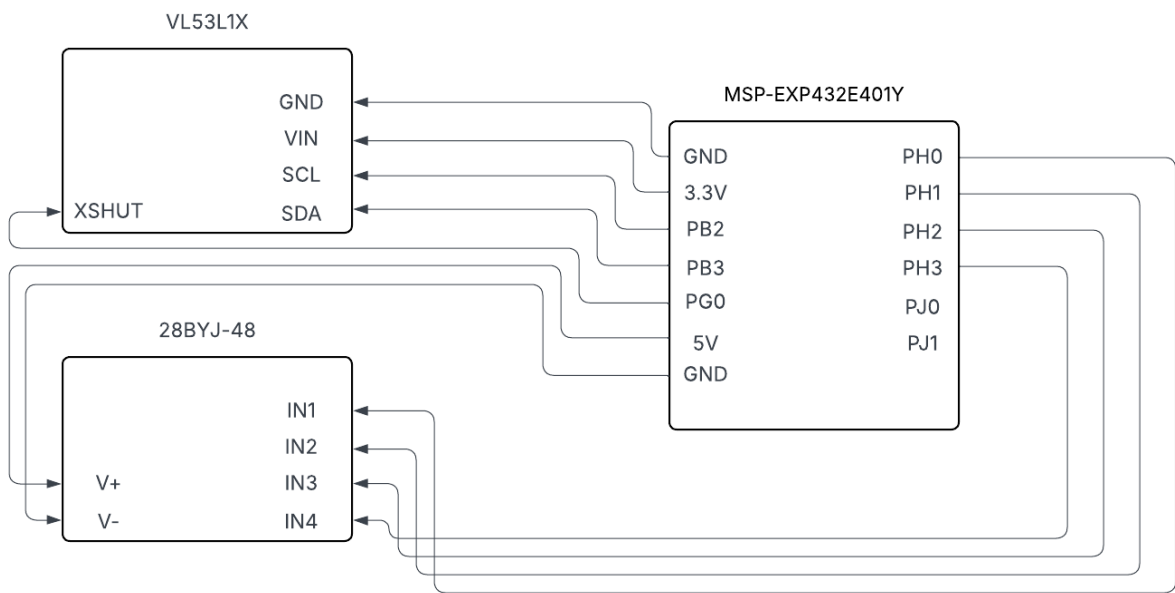
Speed Limitations

- The main limitation of this system is the timingBudget (set to 100ms) and interMeasurementDelay (set to 100ms)
- These can be decreased to gain speed in exchange for accuracy
- If set in short distance mode (about 1.4m max distance), these settings can be reduced to 33ms

Required Bus Speed

- To achieve the bus speed required for optimal system timing which is 22MHz the following values should be used in the PLL
- FXTAL = 25,000,000, Q = 0, N = 4, MINT = 88, PSYSDIV = 19
- This will successfully use the correct clock speed for the program
- Make sure any other timing that uses counters for delay should also be changed to match 22MHz such as SysTick

Circuit Schematic



Programming Logic Flow

